Text Editor Redesign Assignment - Phase 0

Project Overview

This assignment represents Phase 0 of a larger text editor project. In this initial phase, you will focus on redesigning the core data structures and basic functionality of the editor.

Reference Implementation

Before starting your redesign, please review the reference implementation of the text editor available at this link: Text Editor Reference Implementation

This reference code provides a working example of a simple text editor. While its internal structure differs from what you'll be implementing, it serves as a valuable resource for understanding the overall functionality and user interaction of the editor.

Objective

Your task is to redesign and implement a simple text editor inspired by Vim, using specific data structures to manage text content and cursor movement. This assignment will help you practice working with linked lists, vectors, and iterators in C++.

Requirements

1. **Data Structure Design**:
    - Represent each line of text as a linked list, where each node contains a single character.
    - Implement a custom iterator for the line's linked list to facilitate left and right cursor movement within a line.
    - Store all lines in a vector, using the vector's iterator as a line cursor for up and down movement between lines.
2. **Functionality**:
    - Implement basic text editing features: inserting characters, creating new lines, and moving the cursor.
    - Include two modes of operation: Normal mode and Insert mode, similar to Vim.
    - Ensure proper cursor movement using arrow keys in both modes.
3. **Code Structure**:
    - Create a `LinkedList` class to represent a line of text, including its custom iterator.
    - Develop a `TextEditor` class that uses a vector of `LinkedList` objects and implements all editing operations.
4. **User Interface**:
    - Implement a simple console-based interface to display the text and cursor position.
    - Show the current mode (Normal or Insert) to the user.

Using the Reference Code

While redesigning the editor, you may:

- Study the overall structure and logic of the implementation.
- Reuse and adapt the `display()` function for your console output.
- Utilize the `getChar()` function for handling user input, including arrow key detection.
- Reference the logic for cursor movement and mode switching in the main loop.

Learning Objectives

- Gain practical experience in implementing and using custom iterators.
- Understand the application of linked lists and vectors in a real-world scenario.
- Practice designing classes that interact with each other to create a cohesive system.
- Improve skills in adapting and refactoring existing code to meet new requirements.

Submission Guidelines

1. Submit your complete C++ source code files.
2. Include a brief document explaining your design choices and any challenges you faced.
3. Ensure your code is well-commented and follows good coding practices.

Evaluation Criteria

- Correct implementation of the required data structures.
- Proper functionality of text editing features and cursor movement.
- Code quality, including organization, comments, and adherence to C++ best practices.
- Effective use and adaptation of the provided reference code where appropriate.

Remember, this is just the beginning of your text editor project. Future phases may build upon this foundation to add more advanced features and optimizations.

Good luck, and happy coding!